

Informe TFG

Andrés García Treviño

April 2026

1 Revisión del código original

1.1 Capas de la red neuronal

La red neuronal inicial era una red LSTM de 1 capa de 24 neuronas y una capa final de 3 neuronas completamente conectadas (`Dense`).

Para permitir la compatibilidad con otras capas de neuronas y otros entornos de ejecución como Google Colab o Kaggle, he añadido una primera capa de entrada (`Input`) para establecer el tamaño de los *batches* (`batch_input_size`).

```
1 # CNN-LSTM MODEL
2 myCNNLSTM = Sequential()
3 myCNNLSTM.add(Input(batch_shape=(batch_size, look_back, 7, 7, 1)))
4 myCNNLSTM.add(TimeDistributed(Conv2D(filters=current_filters, kernel_size=(3,3),
5 padding='same', activation='relu'))))
6 myCNNLSTM.add(TimeDistributed(Flatten())) # Flatten to connect CNN to LSTM
7 myCNNLSTM.add(LSTM(look_back, stateful=True))
8 myCNNLSTM.add(Dense(num_features))
9 myCNNLSTM.compile(loss='mean_squared_error', optimizer='adam')
```

Listing 1: Capa Input en la red CNN+LSTM

1.2 *Data leakage* por normalización

En el modelo original se utilizan todas las muestras del dataset para generar el objeto `MinMaxScaler` con el que se realiza la normalización de los datos. Esto constituye en sí mismo *data leakage*.

```
1 # normalize the dataset
2 scaler = MinMaxScaler(feature_range=(0, 1))
3 dataset = scaler.fit_transform(dataset)
```

Listing 2: *Data leakage* original

El objetivo del paper SKIMA y de el presente TFG es comparar el entrenamiento incremental *on-line learning* con el entrenamiento estático. Si la red se entrena primeramente pasando 400 veces (`n_epochs`) por el subset de entrenamiento (`train`), el subset de test (`test`) no puede estar incluido en la generación del obketo `sklearn.preprocessing.MinMaxScaler` puesto que la red neuronal estaría preparada de antemano para dicho subset de test y daría unos resultados mejores de los reales.

Además, el entrenamiento de la red neuronal está pensado para desplegar dicha red en un entorno real. En dicho escenario, los datos nuevos que tenga que procesar la red neuronal no podrían generar el objeto `MinMaxScaler`. Si pudieran, aun así habría que reentrenar la red neuronal cada vez que llegara un nuevo dato.

Lo mismo ocurre con el entrenamiento incremental. Está pensado para que, al desplegar la red neuronal en un entorno real, ésta vaya adaptándose a los cambios en los patrones de tráfico, evitando así caer en el *concept drift* y tener que reentrenar la red neuronal. Los nuevos datos que la red tenga que procesar en dicho escenario no van a generar el objeto `MinMaxScaler`, por lo tanto, los datos de test tampoco.

Así, he cambiado el script de python de esta forma:

```

1 dataset = dataset_full[0:train_size+test_size]
2
3 scaler = MinMaxScaler(feature_range=(0, 1))
4 train = dataset[0:train_size, :]
5 test = dataset[train_size : train_size+test_size, :]
6
7 # se escala sin los datos de test. NO DATA LEAKAGE
8 train = scaler.fit_transform(train)
9 test = scaler.transform(test)

```

Listing 3: Escalado del dataset sin data leakage

Los resultados sin *data leakage* corrigen las estimaciones optimistas del test estático con *data leakage*. Esto es porque la red tiene información sobre el set de `test` en su entrenamiento, lo que mejora falsamente la precisión de sus predicciones. Por ello, el RNMSE estático es menor con *data leakage*. Este es el principal argumento para eliminar el *data leakage*.

No ocurre lo mismo para el entrenamiento incremental, que aumenta su precisión sin el *data leakage*. Mi hipótesis es que, como la red deja de tener información previa sobre el conjunto de `test`, la actualización de los pesos es más precisa. Por ejemplo, si el conjunto de `train` tiene un máximo de 1 y el de `test` de 5, con *data leakage* ambos sets estarían divididos por 5 (por el efecto de `MinMaxScaler`). Al llegar al entrenamiento incremental, cuando la red viera dicho máximo de 5, actualizaría sus pesos en menor medida, puesto que todos los pesos han sido entrenados con dicho máximo en mente. Así, los pesos se quedarían "cortos" porque no se actualizarían mucho. Sin *data leakage*, la red tendría sus pesos ajustados al máximo de 1 (del set de `train` de ejemplo) y al ver la muestra del set de `test` con valor 5, realizaría una actualización agresiva de los pesos para ajustarse a dicho máximo.

La explicación anterior también deja entrever un gran problema con el *data leakage*: el entrenamiento de la red con un set de `train` escalado con un mínimo y máximo que no pertenecen a dicho set. Esto provoca que el entrenamiento no ajuste bien los pesos desde un principio.

Table 1: Resultados de la simulación 3 celdas LSTM con *data leakage*

Train RMSE	Static RMSE	Online RMSE	Time (s)
0.1768984	0.193153449	0.211120148	0.144019261
0.256963947	0.314264322	0.261292641	0.144019261
0.186151637	0.221303425	0.22850652	0.144019261

Table 2: Resultados de la simulación 3 celdas LSTM sin *data leakage*

Train RMSE	Static RMSE	Online RMSE	Time (s)
0.173731145	0.195579803	0.207026567	0.047692798
0.260635349	0.305116773	0.249701169	0.047692798
0.186733795	0.229515076	0.215558911	0.047692798

1.3 *Data leakage* por falta de validación

Puede parecer que el sistema de entrenamiento y búsqueda de los hiperparámetros más convenientes para la red neuronal está falto de un set de validación y es cierto. Sin embargo, cuento con que sea posible obtener más datos para generar un subset de test adicional. Por ello, aunque en todos los scripts aparezca el subset de validación nombrado como `test`, sigue siendo validación. Al encontrar la mejor combinación de hiperparámetros, probaré con nuevos datos para comprobar que es una red neuronal viable y no una demasiado adaptada a los datos de validación.

1.4 Reproducibilidad

Con 3 celdas fijar la semilla de Tensorflow es suficiente. Al aumentar a 49 celdas ya no lo es, los procesadores como la GPU utilizan algoritmos de eficiencia en el que reparten la computación por los diferentes hilos y núcleos. Para que mi ordenador personal arrojara resultados reproducibles fijé más semillas y forcé a la GPU a ser determinista. También funciona en Kaggle.

```
1 # fix random seed for reproducibility
2 os.environ['PYTHONHASHSEED'] = '0'
3 os.environ['TF_DETERMINISTIC_OPS'] = '1'
4 os.environ['TF_CUDNN_DETERMINISTIC'] = '1'
5 os.environ['TF_ENABLE_ONEDNN_OPTS'] = '0'
6 random.seed(7)
7 np.random.seed(7)
8 tf.random.set_seed(7)
9 tf.config.experimental.enable_op_determinism()
```

Listing 4: Código para la reproducibilidad

Estas medidas hacen que el entrenamiento sea extremadamente lento puesto que estoy forzado a las CPU y GPUs a ser completamente deterministas, sin que puedan aumentar su eficiencia usando procesamiento en paralelo.

1.5 Optimización del código

1.5.1 Cálculo dinámico del tamaño de los datasets

Como la red neuronal es *stateful*, es decir, se guardan los estados internos de la LSTM de una a otra, el tamaño de los datasets ha de ser divisible entre el tamaño de los *batches*, puesto que keras y tensorflow no permiten que una red LSTM stateful procese un *batch* incompleto. Por ello, he calculado dinámicamente los tamaños de los sets `train` y `test`. El set de `train` se ajusta a ser lo más cercano a 400 muestras, como estaba en el código original. El set de `test` utiliza el resto, siempre siendo divisible por el tamaño del *batch* (`batch_input_size`).

```
1 # SEPARATE INTO TRAIN AND TEST
2 desired_train_size = 400
3 train_available_batches = (desired_train_size - current_look_back - 1) //
  current_batch_size
4 train_size = current_batch_size * train_available_batches + current_look_back + 1
5 #create_dataset se come las primeras look_back + 1 muestras para crear el dataset
  de test/train
6 available_batches = (total_rows - current_look_back - 1 - train_size) //
  current_batch_size
7 test_size = current_batch_size * available_batches + current_look_back + 1
8
9 dataset = dataset_full[0:train_size+test_size]
10 train = dataset[0:train_size, :]
11 test = dataset[train_size : train_size+test_size, :]
```

Listing 5: Adquisición dinámica de datos

1.5.2 Visualización de resultados

Como en una red neuronal que procesa 49 celdas es completamente inviable analizar 49 gráficos como se hacía en la red de 3 celdas original, he decidido crear un *plot* que me indica el RNMSE en cada celda, tanto de entrenamiento, test estático y entrenamiento incremental. Obviamente, esto solo lo implemento cuando pruebo una única combinación de parámetros. También se guardan los resultados numéricos en un fichero Excel.

```
1 # Crear un DataFrame con los resultados
2 region_names = dataframe.columns.tolist()
3 results = {
4     'Region' : region_names,
5     'Train Score': trainScore,
6     'Test Score': testScore,
7     'Test Score incremental': testScore_inc,
8     'Time Score': [timeScore] * num_features # Repetir el valor de timeScore para
9     todas las filas
10 }
11 df = pd.DataFrame(results)
12
13 combined_predictions = np.hstack((test_Y_u[batch_size-1:-1,:], test_Yhat_su[batch_size
14 -1:-1, :], test_Yhat_ou))
15
16 all_cols = [f'Real_{name}' for name in region_names] + \
17             [f'PredStatic_{name}' for name in region_names] + \
18             [f'PredInc_{name}' for name in region_names]
19
20 df2 = pd.DataFrame(combined_predictions, columns=all_cols)
21
22 filename = f'resultados_norandom3_7x7LSTM_LB{look_back}_BS{batch_size}_EU{
23     n_epochs_update}_T{error_threshold}_GWOL'
24 filenameexcel = f'{filename}.xlsx'
25 filenameplot = f'plot{filename}'
26
27 with pd.ExcelWriter(filenameexcel) as writer:
28     df.to_excel(writer, sheet_name='Summary', index=False)
29     df2.to_excel(writer, sheet_name='AllPredictions', index=False)
30
31 plt.figure(figsize=(15,6))
32
33 plt.plot(np.arange(num_features), trainScore, label='Train', marker='o', color='orange')
34 plt.plot(np.arange(num_features), testScore, label='StaticTest', marker='s', color='green')
35 plt.plot(np.arange(num_features), testScore_inc, label='OnlineTest', marker='^', color='firebrick')
36
37 plt.xticks(np.arange(num_features), region_names, rotation=90)
38 plt.tight_layout()
39
40 plt.xlabel('Region')
41 plt.ylabel('RNMSE')
42 plt.title(f'LSTM 7x7 {n_epochs} epochs {n_epochs_update} epoch update')
43 plt.legend(loc='best')
44
45 plt.savefig(filenameplot, dpi=300)
46
47 plt.show()
```

Listing 6: Nuevo método de visualización de resultados

2 Simulaciones

He estado realizando bastantes simulaciones con mi propio ordenador, aprovechando que tiene GPU, para intentar entrenar la redes neuronales. Sin embargo, aunque para una o dos veces esté bien, mi ordenador ha de estar exclusivamente dedicado a ello, cosa que no es posible en muchos casos porque tengo que utilizarlo para otras cosas.

Por ello he probado diferentes plataformas como Google Colab. Ésta es horrible en todos sus sentidos. Sin pagar una millonada apenas te deja hacer nada. Google Colab indicaba que un único entrenamiento de la red 7x7 LSTM tardaba 82 horas. Ni siquiera se podía cerrar la pestaña puesto que el entrenamiento se cortaba y había que volver a empezar.

La última plataforma que he probado es Kaggle, con la que estoy medio contento. Tengo 30 horas gratis de GPU-P-100 a la semana y funciona muy bien. Sin embargo, como ya he comentado, la reproducibilidad es un problema aun habiendo incrementado las medidas para ello.

2.1 Bucle Grid Search

Para evitar ir probando uno a uno los valores de los distintos hiperparámetros que he probado, he creado un bucle que los prueba todos usando la herramienta `itertools`. Comienza definiendo los valores a probar:

```
1 # Parameter tweak for Grid Search
2 hyperparameters = {
3     'look_back': [24],
4     'batch_size': [25],
5     'n_epochs_update': [5, 6, 7],
6     'threshold': [0, 0.05, 0.1],
7     'num_filters': [16, 18, 20]
8 }
9
10 combinations = list(itertools.product(*hyperparameters.values()))
11 output_csv_file = '/kaggle/working/GridSearch_CNNLSTM_Intermediate_v2.csv'
```

Listing 7: Hiperparámetros con `itertools`

Después engloba todo el código, tanto de test estático como de entrenamiento incremental, en un bucle que itera las diferentes combinaciones (La parte del código que no he modificado o que es de menor importancia no la he incluido para no hacer el informe excesivamente largo).

```
1 for combi in combinations:
2     tf.keras.backend.clear_session()
3     tf.random.set_seed(7)
4
5     params = dict(zip(hyperparameters.keys(), combi))
6
7     current_look_back = params['look_back']
8     current_batch_size = params['batch_size']
9     current_n_epochs_update = params['n_epochs_update']
10    current_threshold = params['threshold']
11    current_filters = params['num_filters']
12
13    #GENERAR DATASETS
14
15    # GENERAR MODELO
16    myCNNLSTM = Sequential()
17    myCNNLSTM.add(Input(batch_shape=(batch_size, look_back, 7, 7, 1)))
18    myCNNLSTM.add(TimeDistributed(Conv2D(filters=current_filters, kernel_size=(3,3),
19    padding='same', activation='relu'))
20    myCNNLSTM.add(TimeDistributed(Flatten())) # Flatten to connect CNN to LSTM
21    myCNNLSTM.add(LSTM(look_back, stateful=True))
22    myCNNLSTM.add(Dense(num_features))
23    myCNNLSTM.compile(loss='mean_squared_error', optimizer='adam')
```

```

23
24 # TRAIN MODEL
25 print('Fitting myCNNLSTM on verbose 0')
26 for i in range(n_epochs):
27     myCNNLSTM.fit(trainX, trainY, epochs=1, batch_size=batch_size, verbose=0,
28                   shuffle=True)
29     # Keras 3 Safe Reset
30     for layer in myCNNLSTM.layers:
31         if hasattr(layer, 'reset_states'):
32             layer.reset_states()
33
34 # STATIC TEST
35 train_Y_u = scaler.inverse_transform(trainY)
36 test_Y_u = scaler.inverse_transform(testY)
37
38 train_Yhat_s = myCNNLSTM.predict(trainX, batch_size=batch_size)
39 train_Yhat_su = scaler.inverse_transform(train_Yhat_s)
40
41 test_Yhat_s = myCNNLSTM.predict(testX, batch_size=batch_size)
42 test_Yhat_su = scaler.inverse_transform(test_Yhat_s)
43
44 # ONLINE LEARNING TEST
45 n_epochs_update = current_n_epochs_update
46 error_threshold = current_threshold
47 rmse_norm = 1
48 online_yhat_u = list()
49 times = list()
50 optimizer = tf.keras.optimizers.Adam()
51
52 # Initialize the LSTM with the last train batch (chronological)
53 online_X = trainX[-batch_size:]
54 online_Y = trainY[-batch_size:]
55
56 print('Applying SKIMA backpropagation')
57
58 # BUCLE DE ENTRENAMIENTO INCREMENTAL
59
60 test_Yhat_ou=np.array(online_yhat_u)
61 times=np.array(times)
62
63 #CALCULO DEL RNMSE
64
65 timeScore = np.mean(times)
66 avg_train_score = np.mean(trainScore)
67 avg_test_score = np.mean(testScore)
68 avg_test_score_inc = np.mean(testScore_inc)
69
70 current_result = {
71     # GUARDAR RESULTADOS
72 }
73
74 combi_df = pd.DataFrame([current_result])
75 file_exists = os.path.isfile(output_csv_file)
76 combi_df.to_csv(output_csv_file, mode='a', header=not file_exists, index=False)

```

Listing 8: Bucle Grid Search

2.2 LSTM 3 celdas

Tomando el código original que utilizaba una red LSTM para analizar 3 celdas separadas y suponiendo que los hiperparámetros eran los ideales para esa red y esas celdas, probé a aumentar el número de capas de la red.

También intenté cambiar las funciones de activación pero ello suponía que la GPU no usara los *kernel* cuDNN y la velocidad de entrenamiento caía muchísimo hasta el punto de tardar varias horas en ejecutar el script una sola vez. Lo descarté de inmediato.

Como los resultados no eran muy reproducibles, decidí que, como no estaba haciendo un Grid Search, entrenar varias veces con el mismo número de capas (**super_epochs**) y observar la evolución del error. Así, averigué que, alrededor de 15 o 20 **super_epochs** podía tener suficientes datos para hacer la media del error.

Los resultados fueron fatídicos. Aumentar el número de capas sólo aumentaba el error RNMSE tanto de entrenamiento (**train**), test estático (**test**) y entrenamiento incremental (**test_inc**). Lo mejor era una sola capa de LSTM.

He hecho unos gráficos en Excel para ilustrarlo mejor. Sin embargo, he sido completamente incapaz de cambiar el nombre de los elementos de la leyenda que aparece a la derecha de cada gráfico. Es sencillo identificarlo puesto que la serie2 es 1 capa, la serie3 es 2 capas, etc...

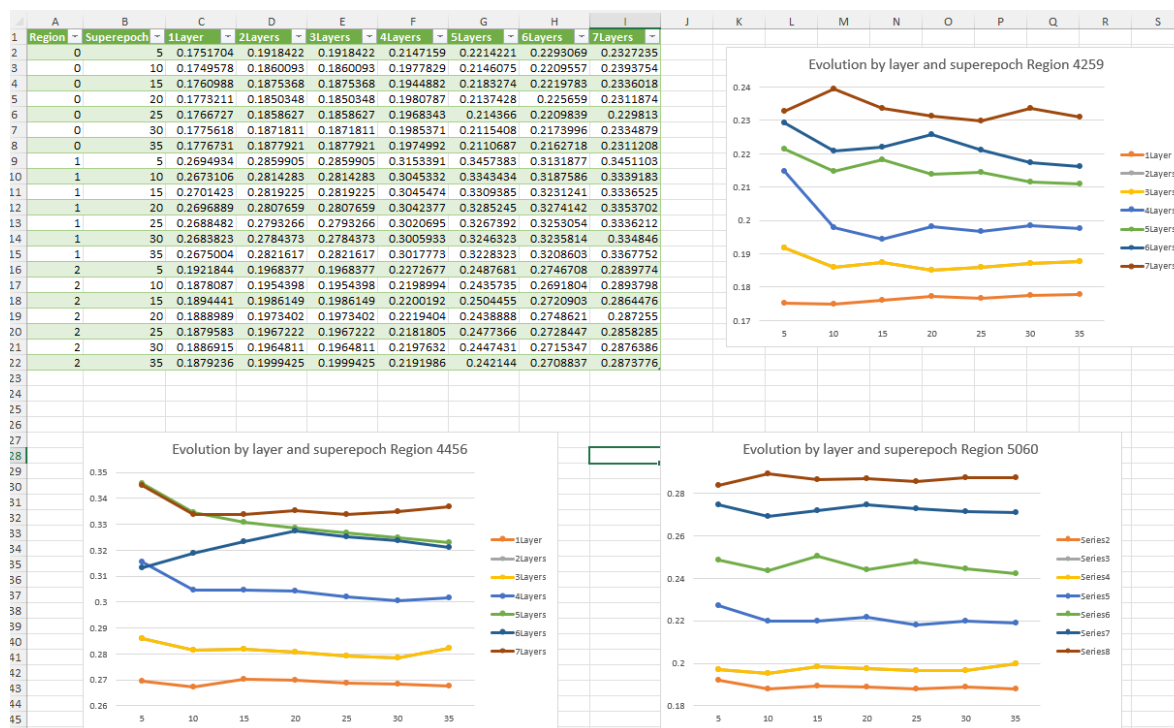


Figure 1: RNMSE de entrenamiento 3 celdas

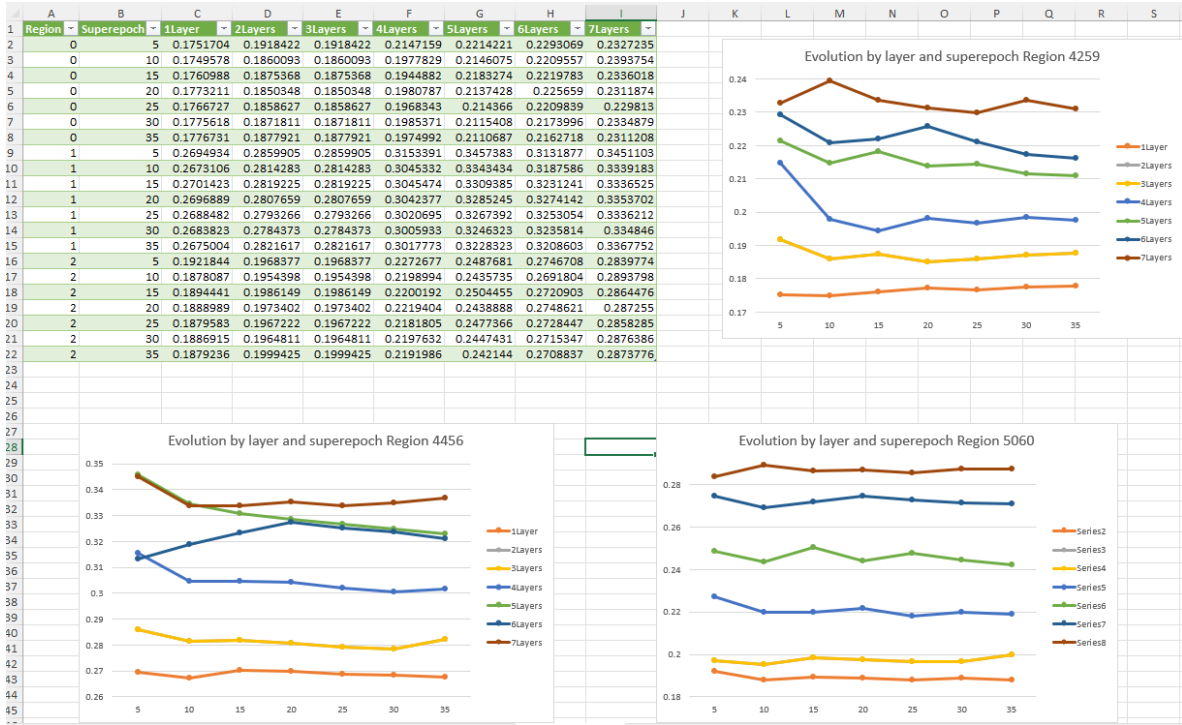


Figure 2: RNMSE de test estático 3 celdas

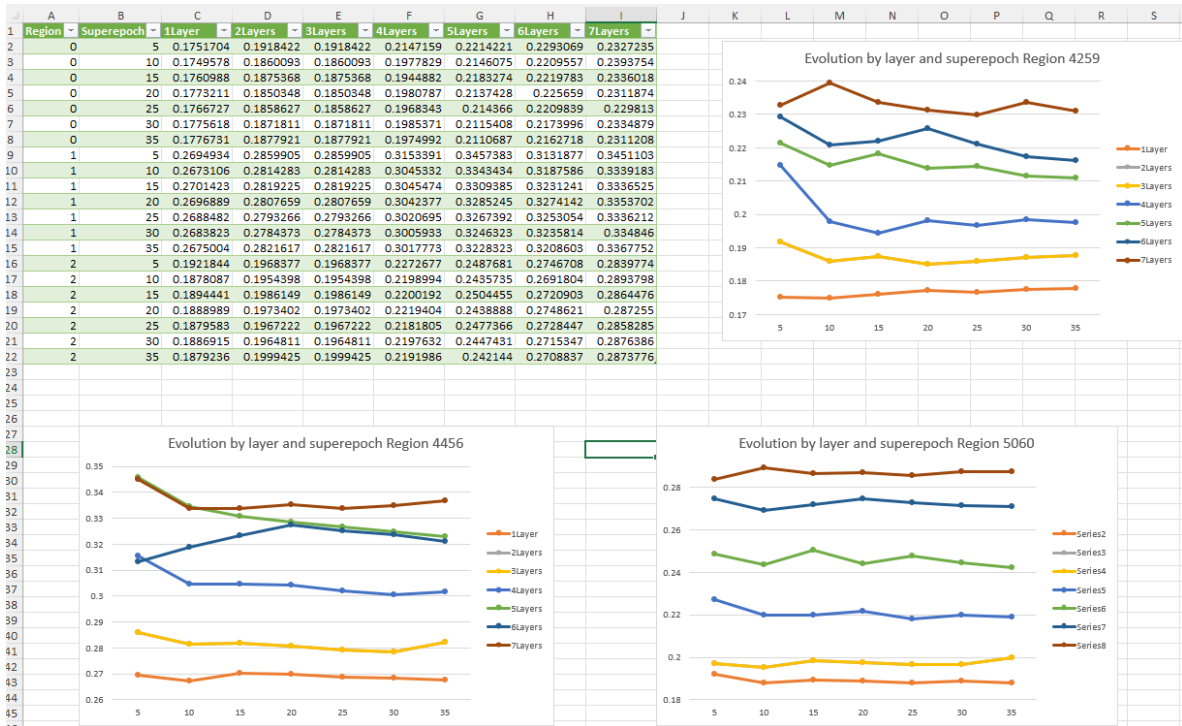


Figure 3: RNMSE de entrenamiento incremental 3 celdas

2.3 7x7 LSTM

El TFG está basado en el uso de una red convolucional (CNN) en una zona de 49 celdas de tráfico dispuestas en un cuadrado de 7x7 celdas contiguas. Por ello he decidido no pasar mucho tiempo

entrenando una red neuronal de 3 celdas separadas entre sí.

Primeramente he entrenado una red neuronal compuesta solamente por una capa de LSTM, incluyendo claramente una capa de entrada (`Input`) y una de salida (`Dense`).

```
1 myLSTM = Sequential()
2 myLSTM.add(Input(batch_shape=(batch_size, trainX.shape[1], trainX.shape[2])))
3 myLSTM.add(LSTM(look_back, stateful=True))
4 myLSTM.add(Dense(trainX.shape[2]))
5 myLSTM.compile(loss='mean_squared_error', optimizer='adam')
6 myLSTM.summary()
```

Listing 9: Capas de la red 7x7LSTM

2.3.1 Resultados

Por ahora, sin haber conseguido reproducibilidad, he ejecutado el Grid Search 3 veces. Cuando los ejecuté pensé que sería reproducible, por lo que en la segunda y tercera ejecución usé los hiperparámetros que mejor lo hicieron en las anteriores ejecuciones. Para evaluarlas, calculé la media del RNMSE de cada celda en cada combinación de hiperparámetros y comparé el error en el entrenamiento incremental, que es el que interesa para el TFG.

La primera ejecución busqué afinar el tamaño de la ventana `lookback` y el tamaño del `batch`, siendo los mejores valores `lookback=12` y 24 horas y `batchsize=25`. También descarté actualizar los pesos sólo 3 veces con cada nuevo dato (`n_epoch_updates`) y usar un `threshold` mayor al 20%.

En la segunda ví que la ventana `lookback` se podía reducir a 16 y 20 horas, actualizar los pesos entre 6 y 7 veces con cada nuevo dato y usar un `threshold` entre 12 y 14%.

En la tercera probé a afinar el tamaño del `batch`, consiguiendo el óptimo entre 22 y 23 ventanas.

Con dichos resultados, como varias combinaciones de parámetros estaban repetidas entre una y otra ejecución, pude comprobar que no eran resultados reproducibles. La diferencia de resultados era de incluso el 10% de RNMSE, lo que es absolutamente inaceptable.

2.4 7x7 CNN-LSTM

Después he añadido una capa de CNN para extraer la correlación espacial entre las celdas y así la capa LSTM tenga más información para realizar sus predicciones.

Para conectar la red CNN con la red LSTM he introducido una capa de tipo `Flatten`, que cambia el formato de la salida de la CNN para adaptarlo a la entrada de la LSTM. La CNN produce una salida del tipo (*width, height, filters*) pero la LSTM requiere un vector de entrada en una dimensión. La capa `Flatten` no aprende nada ni ajusta pesos, sólo cambia la forma de los vectores.

```
1
2
3 # CNN-LSTM MODEL (Keras 3 Compliant)
4 myCNNLSTM = Sequential()
5 myCNNLSTM.add(Input(batch_shape=(batch_size, look_back, 7, 7, 1)))
6 myCNNLSTM.add(TimeDistributed(Conv2D(filters=current_filters, kernel_size=(3,3),
7 padding='same', activation='relu'))))
8 myCNNLSTM.add(TimeDistributed(Flatten())) # Flatten to connect CNN to LSTM
9 myCNNLSTM.add(LSTM(look_back, stateful=True))
10 myCNNLSTM.add(Dense(num_features))
11 myCNNLSTM.compile(loss='mean_squared_error', optimizer='adam')
```

Listing 10: Añadido `enable_op.determinism()`

He descartado con unas simulaciones iniciales y con un poco de lógica aumentar el número de capas. La red es perfectamente capaz de seguir los patrones con una sola capa LSTM.

2.4.1 Resultados

Tras ejecutar varias veces el script de Grid Search no parece que haya una mejoría notable en los resultados de la red neuronal CNLSTM frente a sólo LSTM. Además, el tiempo de procesado aumenta bastante y por si no fuera poco, los resultados no son reproducibles.

Como en la red 7x7LSTM, he intentado ir afinando los hiperparámetros: una ventana de 24 horas, *batches* de 25 ventanas, *threshold* del 5%, *n_epoch_updates* entre 5 y 7 y entre 16 y 20 filtros para la capa convolucional.

3 Simulaciones con reproducibilidad

3.1 Resultados 7x7 LSTM

He utilizado un Grid Search con pocos valores puesto que era una prueba de reproducibilidad y suponía que la velocidad de procesamiento se vería afectada (alrededor de 8 horas de procesado).

He probado los siguientes valores:

```
1 #Parameter tweak
2 hyperparameters = {
3     'look_back': [16, 18],
4     'batch_size': [22, 23, 24],
5     'n_epochs_update': [6, 7],
6     'threshold': [0.13, 0.15],
7 }
```

Listing 11: Grid Search LSTM reproducible

Dichos valores están sacados de las pruebas que realicé cuando el código no era reproducible. Los resultados se encuentran en las Tablas 3 y 4. Como se puede observar, lo único que varía es el tiempo medio que tarda la red en procesar un *batch* en el entrenamiento incremental.

Ahora que sé que es reproducible, volveré a probar otros valores para tener las medidas correctas.

Table 3: Resultados de la simulación n^o1 7x7 LSTM (LB=*lookback*
BS=*batchsize* NEU=*n_epoch_updates* TH=*threshold*)

LB	BS	NEU	TH	Train	Static	Online	Time
16	22	6	0.13	0.154410219	0.315033616	0.307496105	0.019918673
16	22	6	0.15	0.154410219	0.315033616	0.249393058	0.020142126
16	22	7	0.13	0.154410219	0.315033616	0.278063565	0.023258013
16	22	7	0.15	0.154410219	0.315033616	0.326107214	0.023298333
16	23	6	0.13	0.152955137	0.308717743	0.272470725	0.020067559
16	23	6	0.15	0.152955137	0.308717743	0.273582573	0.020229155
16	23	7	0.13	0.152955137	0.308717743	0.264865823	0.024041752
16	23	7	0.15	0.152955137	0.308717743	0.327777837	0.023811805
16	24	6	0.13	0.153923372	0.332922161	0.321783011	0.020786237
16	24	6	0.15	0.153923372	0.332922161	0.303317086	0.020903767
16	24	7	0.13	0.153923372	0.332922161	0.298749961	0.023891294

LB	BS	NEU	TH	Train	Static	Online	Time
16	24	7	0.15	0.153923372	0.332922161	0.254875113	0.023715
18	22	6	0.13	0.148488336	0.308353471	0.525646919	0.021442422
18	22	6	0.15	0.148488336	0.308353471	0.343049847	0.021439737
18	22	7	0.13	0.148488336	0.308353471	0.234839209	0.024486955
18	22	7	0.15	0.148488336	0.308353471	0.225091785	0.024609919
18	23	6	0.13	0.150326923	0.318820435	0.323809765	0.021385165
18	23	6	0.15	0.150326923	0.318820435	0.42029475	0.021450469
18	23	7	0.13	0.150326923	0.318820435	0.277286021	0.02431804
18	23	7	0.15	0.150326923	0.318820435	0.293741878	0.024218096
18	24	6	0.13	0.153354797	0.303344994	0.235961568	0.020987318
18	24	6	0.15	0.153354797	0.303344994	0.253983694	0.021023506
18	24	7	0.13	0.153354797	0.303344994	0.366032881	0.024080219
18	24	7	0.15	0.153354797	0.303344994	0.272740438	0.024510566

Table 4: Resultados de la simulación n°2 7x7 LSTM (LB=*lookback*
BS=*batchsize* NEU=*n_epoch_updates* TH=*threshold*)

LB	BS	NEU	TH	Train	Static	Online	Time
16	22	6	0.13	0.154410219	0.315033616	0.307496105	0.024714466
16	22	6	0.15	0.154410219	0.315033616	0.249393058	0.024725697
16	22	7	0.13	0.154410219	0.315033616	0.278063565	0.028084389
16	22	7	0.15	0.154410219	0.315033616	0.326107214	0.028214874
16	23	6	0.13	0.152955137	0.308717743	0.272470725	0.024666065
16	23	6	0.15	0.152955137	0.308717743	0.273582573	0.024710741
16	23	7	0.13	0.152955137	0.308717743	0.264865823	0.028614211
16	23	7	0.15	0.152955137	0.308717743	0.327777837	0.028256213
16	24	6	0.13	0.153923372	0.332922161	0.321783011	0.02473869
16	24	6	0.15	0.153923372	0.332922161	0.303317086	0.024790993
16	24	7	0.13	0.153923372	0.332922161	0.298749961	0.02801288
16	24	7	0.15	0.153923372	0.332922161	0.254875113	0.028845822
18	22	6	0.13	0.148488336	0.308353471	0.525646919	0.025687575
18	22	6	0.15	0.148488336	0.308353471	0.343049847	0.025913707
18	22	7	0.13	0.148488336	0.308353471	0.234839209	0.029606785
18	22	7	0.15	0.148488336	0.308353471	0.225091785	0.029639727
18	23	6	0.13	0.150326923	0.318820435	0.323809765	0.025859383
18	23	6	0.15	0.150326923	0.318820435	0.42029475	0.025919308
18	23	7	0.13	0.150326923	0.318820435	0.277286021	0.027593746
18	23	7	0.15	0.150326923	0.318820435	0.293741878	0.026875117
18	24	6	0.13	0.153354797	0.303344994	0.235961568	0.02340933
18	24	6	0.15	0.153354797	0.303344994	0.253983694	0.02423076
18	24	7	0.13	0.153354797	0.303344994	0.366032881	0.028530059
18	24	7	0.15	0.153354797	0.303344994	0.272740438	0.029444532

Los 3 mejores resultados han sido con `lookback=18h`, `batch_size=[23,24]`, `n_epoch_updates=[6,7]` y `threshold=0.13`.

En las combinaciones ganadoras el test estático ha tenido un 10% más de RNMSE que el entrenamiento incremental, estando el test estático alrededor de 0.31 y el entrenamiento incremental alrededor de 0.23.

Sin embargo, el entrenamiento incremental tiene una varianza mayor a lo largo de las diferentes combinaciones, variando entre 0.21 y 0.518, mientras el test estático se mantiene prácticamente constante independientemente de los hiperparámetros.

3.2 Resultados CNN+LSTM

También he utilizado un Grid Search con pocos valores sacados de las simulaciones no reproducibles puesto que era una prueba de reproducibilidad y suponía que la velocidad de procesamiento se vería afectada (alrededor de 17 horas de procesado).

He probado los siguientes valores:

```

1 #Parameter tweak
2 hyperparameters = {
3     'look_back': [12, 24],
4     'batch_size': [20, 25],
5     'n_epochs_update': [6, 7],
6     'threshold': [0, 0.1],
7     'num_filters': [16, 20]
8 }

```

Listing 12: Grid Search CNN+LSTM reproducible

Los resultados son reproducibles pero no muy esperanzadores, mostrados en las Tablas 5 y 6. He conseguido la reproducibilidad ya que lo único que varía es el tiempo de procesado medio de un *batch* en el entrenamiento incremental.

Table 5: Resultados de la simulación n^o1 7x7 CNN+LSTM
(LB=*lookback* BS=*batchsize* NEU=*n_epoch_updates* TH=*threshold*
NF=*número filtros*)

LB	BS	NEU	TH	NF	Train	Static	Online	Time
12	20	6	0	16	0.136435987	0.336438593	0.309519549	0.775301229
12	20	6	0	20	0.135498539	0.325804514	0.283527423	0.781537186
12	20	6	0.1	16	0.136435987	0.336438593	0.300620419	0.776164829
12	20	6	0.1	20	0.135498539	0.325804514	0.262248954	0.784402296
12	20	7	0	16	0.136435987	0.336438593	0.37988848	0.900083346
12	20	7	0	20	0.135498539	0.325804514	0.384300649	0.903238656
12	20	7	0.1	16	0.136435987	0.336438593	0.316892445	0.897863784
12	20	7	0.1	20	0.135498539	0.325804514	0.288187836	0.901250708
12	25	6	0	16	0.140831023	0.332049578	0.434571621	0.782668529
12	25	6	0	20	0.140729277	0.318985044	0.276990388	0.788760814
12	25	6	0.1	16	0.140831023	0.332049578	0.285691321	0.78126786
12	25	6	0.1	20	0.140729277	0.318985044	0.263184685	0.793162529
12	25	7	0	16	0.140831023	0.332049578	0.286407941	0.909946667
12	25	7	0	20	0.140729277	0.318985044	0.257898167	0.921776949
12	25	7	0.1	16	0.140831023	0.332049578	0.298456066	0.914204495
12	25	7	0.1	20	0.140729277	0.318985044	0.301554435	0.898031408
24	20	6	0	16	0.117808723	0.340731834	0.434977389	1.095122286
24	20	6	0	20	0.119970717	0.325216536	0.400376778	1.092309314
24	20	6	0.1	16	0.117808723	0.340731834	0.274862862	1.078515166
24	20	6	0.1	20	0.119970717	0.325216536	0.760181228	1.102018269
24	20	7	0	16	0.117808723	0.340731834	0.272577592	1.266302368
24	20	7	0	20	0.119970717	0.325216536	0.373406402	1.243465392
24	20	7	0.1	16	0.117808723	0.340731834	0.303903959	1.28417865
24	20	7	0.1	20	0.119970717	0.325216536	0.33642228	1.260708087

LB	BS	NEU	TH	NF	Train	Static	Online	Time
24	25	6	0	16	0.118254842	0.347724685	0.36746318	1.096795116
24	25	6	0	20	0.122224145	0.317708799	0.239031252	1.105234044
24	25	6	0.1	16	0.118254842	0.347724685	0.3642279	1.114422824
24	25	6	0.1	20	0.122224145	0.317708799	0.3629521	1.132336129
24	25	7	0	16	0.118254842	0.347724685	0.281169608	1.289590618
24	25	7	0	20	0.122224145	0.317708799	0.321650808	1.253162395
24	25	7	0.1	16	0.118254842	0.347724685	0.221318145	1.250830714
24	25	7	0.1	20	0.122224145	0.317708799	0.249178129	1.26143209

Table 6: Resultados de la simulación n^o2 7x7 CNN+LSTM
(LB=*lookback* BS=*batchsize* NEU=*n_epoch_updates* TH=*threshold*
NF=*número filtros*)

LB	BS	NEU	TH	NF	Train	Static	Online	Time
12	20	6	0	16	0.136435987	0.336438593	0.309519549	0.626638152
12	20	6	0	20	0.135498539	0.325804514	0.283527423	0.627031923
12	20	6	0.1	16	0.136435987	0.336438593	0.300620419	0.612605933
12	20	6	0.1	20	0.135498539	0.325804514	0.262248954	0.61318003
12	20	7	0	16	0.136435987	0.336438593	0.37988848	0.692452011
12	20	7	0	20	0.135498539	0.325804514	0.384300649	0.693050666
12	20	7	0.1	16	0.136435987	0.336438593	0.316892445	0.689793333
12	20	7	0.1	20	0.135498539	0.325804514	0.288187836	0.700007479
12	25	6	0	16	0.140831023	0.332049578	0.434571621	0.603255661
12	25	6	0	20	0.140729277	0.318985044	0.276990388	0.602316875
12	25	6	0.1	16	0.140831023	0.332049578	0.285691321	0.60376873
12	25	6	0.1	20	0.140729277	0.318985044	0.263184685	0.606039568
12	25	7	0	16	0.140831023	0.332049578	0.286407941	0.709586567
12	25	7	0	20	0.140729277	0.318985044	0.257898167	0.737659439
12	25	7	0.1	16	0.140831023	0.332049578	0.298456066	0.72931029
12	25	7	0.1	20	0.140729277	0.318985044	0.301554435	0.748357474
24	20	6	0	16	0.117808723	0.340731834	0.434977389	0.933427418
24	20	6	0	20	0.119970717	0.325216536	0.400376778	1.066987046
24	20	6	0.1	16	0.117808723	0.340731834	0.274862862	1.072112396
24	20	6	0.1	20	0.119970717	0.325216536	0.760181228	1.064668029
24	20	7	0	16	0.117808723	0.340731834	0.272577592	1.240352885
24	20	7	0	20	0.119970717	0.325216536	0.373406402	1.241354773
24	20	7	0.1	16	0.117808723	0.340731834	0.303903959	1.247211036
24	20	7	0.1	20	0.119970717	0.325216536	0.33642228	1.245750006
24	25	6	0	16	0.118254842	0.347724685	0.36746318	1.068491374
24	25	6	0	20	0.122224145	0.317708799	0.239031252	1.071433731
24	25	6	0.1	16	0.118254842	0.347724685	0.3642279	1.050309789
24	25	6	0.1	20	0.122224145	0.317708799	0.3629521	1.058564282
24	25	7	0	16	0.118254842	0.347724685	0.281169608	1.226235943
24	25	7	0	20	0.122224145	0.317708799	0.321650808	1.233290489
24	25	7	0.1	16	0.118254842	0.347724685	0.221318145	1.239624583
24	25	7	0.1	20	0.122224145	0.317708799	0.249178129	1.244544553

Los 3 mejores resultados fueron con `lookback=24h`, `batch_size=25`, `n_epochs_update=[6,7]`, `threshold=[0,0.1]` y entre 16 y 20 filtros. Los resultados no han supuesto una mejora respecto a la red neuronal

LSTM. El valor del RNMSE de CNN+LSTM en cuanto a entrenamiento incremental es muy parecido sin llegar a superar el de LSTM.

Cabe destacar que con la red CNN el RNMSE del entrenamiento incremental es más estable entre combinaciones de hiperparámetros que con sólo la red LSTM.

4 Grid Search extendido con aumento de velocidad

He realizado un Grid Search más exhaustivo, detallado en la Tabla 7. Como la reproducibilidad ya está implementada, este Grid Search no contiene las combinaciones de las Tablas 5 y 6.

Los mejores resultados han sido: *lookback*=24, aunque *lookback*=18 parece ser mejor que *lookback*=12; *batchsize*=23, *n_epoch_updates*=6, *threshold*=10%, destacando que la ausencia de *threshold* también da muy buenos resultados independientemente del resto de hiperparámetros y un *threshold* demasiado elevado no es viable; *num_filters*=16.

Table 7: Resultados de la simulación con Grid Search extendido 7x7
CNN+LSTM y mejoras de velocidad (LB=*lookback* BS=*batchsize*
NEU=*n_epoch_updates* TH=*threshold* NF=número filtros)

LB	BS	NEU	TH	NF	Train	Static	Online	Time
12	20	5	0	16	0.136435987	0.336438593	0.315445888	0.02757469
12	20	5	0	18	0.138510311	0.326107852	0.413911498	0.028649796
12	20	5	0	20	0.135498539	0.325804514	0.354485161	0.027686205
12	20	5	0.1	16	0.136435987	0.336438593	0.488243405	0.027484346
12	20	5	0.1	18	0.138510311	0.326107852	0.410907713	0.028399394
12	20	5	0.1	20	0.135498539	0.325804514	0.335227716	0.02800322
12	20	5	0.5	16	0.136435987	0.336438593	0.424276243	0.033062468
12	20	5	0.5	18	0.138510311	0.326107852	0.319070671	0.033434413
12	20	5	0.5	20	0.135498539	0.325804514	0.309906383	0.050293431
12	20	6	0	18	0.138510311	0.326107852	0.306620305	0.033249064
12	20	6	0.1	18	0.138510311	0.326107852	0.369522493	0.033302386
12	20	6	0.5	16	0.136435987	0.336438593	0.349864357	0.035956842
12	20	6	0.5	18	0.138510311	0.326107852	0.395449133	0.037378202
12	20	6	0.5	20	0.135498539	0.325804514	0.28763296	0.039794498
12	20	7	0	18	0.138510311	0.326107852	0.498220386	0.038639605
12	20	7	0.1	18	0.138510311	0.326107852	0.266783037	0.038509295
12	20	7	0.5	16	0.136435987	0.336438593	0.282705025	0.045757024
12	20	7	0.5	18	0.138510311	0.326107852	0.35248547	0.044054232
12	20	7	0.5	20	0.135498539	0.325804514	0.323421081	0.048680628
12	23	5	0	16	0.163683806	0.310488177	0.25276199	0.028206747
12	23	5	0	18	0.191792312	0.313789065	0.339138751	0.029220118
12	23	5	0	20	0.149129542	0.310096554	0.454356282	0.028102564
12	23	5	0.1	16	0.163683806	0.310488177	0.448968258	0.028001268
12	23	5	0.1	18	0.191792312	0.313789065	0.466502204	0.028728778
12	23	5	0.1	20	0.149129542	0.310096554	0.272141155	0.029016897
12	23	5	0.5	16	0.163683806	0.310488177	0.361072925	0.037107385
12	23	5	0.5	18	0.191792312	0.313789065	0.355444945	0.034520278
12	23	5	0.5	20	0.149129542	0.310096554	0.364362493	0.046756702
12	23	6	0	16	0.163683806	0.310488177	0.371240712	0.0334291
12	23	6	0	18	0.191792312	0.313789065	0.440105914	0.033698077
12	23	6	0	20	0.149129542	0.310096554	0.333190807	0.033899505
12	23	6	0.1	16	0.163683806	0.310488177	0.530360013	0.033133841

LB	BS	NEU	TH	NF	Train	Static	Online	Time
12	23	6	0.1	18	0.191792312	0.313789065	0.256510533	0.033913294
12	23	6	0.1	20	0.149129542	0.310096554	0.537276387	0.034126491
12	23	6	0.5	16	0.163683806	0.310488177	0.297694435	0.038356194
12	23	6	0.5	18	0.191792312	0.313789065	0.341480768	0.039937094
12	23	6	0.5	20	0.149129542	0.310096554	0.350884832	0.046619215
12	23	7	0	16	0.163683806	0.310488177	0.25660396	0.03884932
12	23	7	0	18	0.191792312	0.313789065	0.25641798	0.039045478
12	23	7	0	20	0.149129542	0.310096554	0.533563212	0.039312762
12	23	7	0.1	16	0.163683806	0.310488177	0.361071249	0.038909419
12	23	7	0.1	18	0.191792312	0.313789065	0.253997468	0.03941358
12	23	7	0.1	20	0.149129542	0.310096554	0.350028629	0.039502083
12	23	7	0.5	16	0.163683806	0.310488177	0.271988095	0.04592449
12	23	7	0.5	18	0.191792312	0.313789065	0.278681715	0.042506903
12	23	7	0.5	20	0.149129542	0.310096554	0.358859532	0.04174355
12	25	5	0	16	0.140831023	0.332049578	0.304053971	0.028388267
12	25	5	0	18	0.140274747	0.339992052	0.471792821	0.029790833
12	25	5	0	20	0.140729277	0.318985044	0.295276939	0.02933928
12	25	5	0.1	16	0.140831023	0.332049578	0.374727835	0.02936971
12	25	5	0.1	18	0.140274747	0.339992052	0.639532086	0.029768318
12	25	5	0.1	20	0.140729277	0.318985044	0.348532402	0.029584602
12	25	5	0.5	16	0.140831023	0.332049578	0.34816462	0.032745421
12	25	5	0.5	18	0.140274747	0.339992052	0.310210295	0.035851433
12	25	5	0.5	20	0.140729277	0.318985044	0.32780507	0.032121062
12	25	6	0	18	0.140274747	0.339992052	0.453894436	0.033884039
12	25	6	0.1	18	0.140274747	0.339992052	1.301283526	0.033482335
12	25	6	0.5	16	0.140831023	0.332049578	0.289479545	0.039108899
12	25	6	0.5	18	0.140274747	0.339992052	0.344352871	0.046676698
12	25	6	0.5	20	0.140729277	0.318985044	0.370562344	0.038727413
12	25	7	0	18	0.140274747	0.339992052	0.343432875	0.039021849
12	25	7	0.1	18	0.140274747	0.339992052	0.314101687	0.039221571
12	25	7	0.5	16	0.140831023	0.332049578	0.400846216	0.040453789
12	25	7	0.5	18	0.140274747	0.339992052	0.299707008	0.044716686
12	25	7	0.5	20	0.140729277	0.318985044	0.29200197	0.049683946
18	20	5	0	16	0.124188291	0.351130706	0.44549564	0.032867833
18	20	5	0	18	0.125561637	0.34379394	0.279698489	0.033507451
18	20	5	0	20	0.157130497	0.333308253	0.289715531	0.032463338
18	20	5	0.1	16	0.124188291	0.351130706	0.337810675	0.030352295
18	20	5	0.1	18	0.125561637	0.34379394	0.324740071	0.030957301
18	20	5	0.1	20	0.157130497	0.333308253	0.557908275	0.030610056
18	20	5	0.5	16	0.124188291	0.351130706	0.261651886	0.040797204
18	20	5	0.5	18	0.125561637	0.34379394	0.317533034	0.03606078
18	20	5	0.5	20	0.157130497	0.333308253	0.280227668	0.039849146
18	20	6	0	16	0.124188291	0.351130706	0.22750189	0.035662545
18	20	6	0	18	0.125561637	0.34379394	0.330851318	0.035961099
18	20	6	0	20	0.157130497	0.333308253	0.279271202	0.035047249
18	20	6	0.1	16	0.124188291	0.351130706	0.459342456	0.034787498
18	20	6	0.1	18	0.125561637	0.34379394	0.401013509	0.035679612
18	20	6	0.1	20	0.157130497	0.333308253	0.299093759	0.035048685
18	20	6	0.5	16	0.124188291	0.351130706	0.303918994	0.046009271
18	20	6	0.5	18	0.125561637	0.34379394	0.39852485	0.038462612
18	20	6	0.5	20	0.157130497	0.333308253	0.263999611	0.045986775
18	20	7	0	16	0.124188291	0.351130706	0.270392779	0.039796924
18	20	7	0	18	0.125561637	0.34379394	0.36236949	0.040383231
18	20	7	0	20	0.157130497	0.333308253	0.264339883	0.039690626
18	20	7	0.1	16	0.124188291	0.351130706	0.328284274	0.040293827

LB	BS	NEU	TH	NF	Train	Static	Online	Time
18	20	7	0.1	18	0.125561637	0.34379394	0.242497093	0.040590078
18	20	7	0.1	20	0.157130497	0.333308253	0.342248843	0.039974868
18	20	7	0.5	16	0.124188291	0.351130706	0.275108437	0.045760756
18	20	7	0.5	18	0.125561637	0.34379394	0.451926269	0.042284695
18	20	7	0.5	20	0.157130497	0.333308253	0.276592488	0.04984677
18	23	5	0	16	0.132810299	0.33521819	0.367564029	0.030673811
18	23	5	0	18	0.128073586	0.32483796	0.615401825	0.030784797
18	23	5	0	20	0.126368559	0.317677541	0.472691303	0.030580541
18	23	5	0.1	16	0.132810299	0.33521819	0.27981442	0.029565398
18	23	5	0.1	18	0.128073586	0.32483796	0.326615437	0.03059983
18	23	5	0.1	20	0.126368559	0.317677541	0.408602846	0.030408025
18	23	5	0.5	16	0.132810299	0.33521819	0.355172789	0.03606778
18	23	5	0.5	18	0.128073586	0.32483796	0.407765671	0.034781146
18	23	5	0.5	20	0.126368559	0.317677541	0.274672361	0.035647229
18	23	6	0	16	0.132810299	0.33521819	0.286590484	0.035219487
18	23	6	0	18	0.128073586	0.32483796	0.273141048	0.03532709
18	23	6	0	20	0.126368559	0.317677541	0.504135745	0.035575443
18	23	6	0.1	16	0.132810299	0.33521819	0.268178265	0.03479157
18	23	6	0.1	18	0.128073586	0.32483796	0.293402615	0.035651929
18	23	6	0.1	20	0.126368559	0.317677541	0.246195606	0.035623868
18	23	6	0.5	16	0.132810299	0.33521819	0.334052626	0.037902778
18	23	6	0.5	18	0.128073586	0.32483796	0.297844432	0.039584539
18	23	6	0.5	20	0.126368559	0.317677541	0.284678399	0.044010284
18	23	7	0	16	0.132810299	0.33521819	0.261624957	0.040276404
18	23	7	0	18	0.128073586	0.32483796	0.28099613	0.04223907
18	23	7	0	20	0.126368559	0.317677541	0.247007135	0.043097759
18	23	7	0.1	16	0.132810299	0.33521819	0.311369425	0.043027274
18	23	7	0.1	18	0.128073586	0.32483796	0.323032644	0.044723524
18	23	7	0.1	20	0.126368559	0.317677541	0.303077048	0.044731894
18	23	7	0.5	16	0.132810299	0.33521819	0.336221312	0.051536809
18	23	7	0.5	18	0.128073586	0.32483796	0.409013863	0.050712824
18	23	7	0.5	20	0.126368559	0.317677541	0.452651562	0.045065927
18	25	5	0	16	0.140966651	0.317910143	0.279664385	0.032794129
18	25	5	0	18	0.125616867	0.32494159	0.300975342	0.033728964
18	25	5	0	20	0.134359081	0.3193303	0.452861752	0.032525784
18	25	5	0.1	16	0.140966651	0.317910143	0.244987354	0.032419967
18	25	5	0.1	18	0.125616867	0.32494159	0.262777857	0.033501401
18	25	5	0.1	20	0.134359081	0.3193303	0.246148344	0.033602963
18	25	5	0.5	16	0.140966651	0.317910143	0.271096583	0.037249161
18	25	5	0.5	18	0.125616867	0.32494159	0.332152581	0.035403193
18	25	5	0.5	20	0.134359081	0.3193303	0.347808418	0.034517668
18	25	6	0	16	0.140966651	0.317910143	0.304314092	0.038344273
18	25	6	0	18	0.125616867	0.32494159	0.227776985	0.03970774
18	25	6	0	20	0.134359081	0.3193303	0.460571397	0.039773329
18	25	6	0.1	16	0.140966651	0.317910143	0.279490356	0.037643218
18	25	6	0.1	18	0.125616867	0.32494159	0.32650844	0.038279126
18	25	6	0.1	20	0.134359081	0.3193303	0.327456883	0.038435166
18	25	6	0.5	16	0.140966651	0.317910143	0.274607351	0.041310362
18	25	6	0.5	18	0.125616867	0.32494159	0.273795625	0.040855898
18	25	6	0.5	20	0.134359081	0.3193303	0.29187384	0.045863802
18	25	7	0	16	0.140966651	0.317910143	0.269967791	0.042763462
18	25	7	0	18	0.125616867	0.32494159	0.259485365	0.043026591
18	25	7	0	20	0.134359081	0.3193303	0.274488607	0.049615151
18	25	7	0.1	16	0.140966651	0.317910143	0.278330354	0.044159557
18	25	7	0.1	18	0.125616867	0.32494159	0.311502188	0.047806987

LB	BS	NEU	TH	NF	Train	Static	Online	Time
18	25	7	0.1	20	0.134359081	0.3193303	0.33766317	0.050050353
18	25	7	0.5	16	0.140966651	0.317910143	0.267004107	0.055928974
18	25	7	0.5	18	0.125616867	0.32494159	0.299357134	0.052741786
18	25	7	0.5	20	0.134359081	0.3193303	0.348501788	0.0538495
24	20	5	0	16	0.117808723	0.340731834	0.393406854	0.037318848
24	20	5	0	18	0.112246438	0.35286297	0.24014195	0.037610687
24	20	5	0	20	0.119970717	0.325216536	0.276168363	0.036914802
24	20	5	0.1	16	0.117808723	0.340731834	0.390218603	0.037134678
24	20	5	0.1	18	0.112246438	0.35286297	0.312101398	0.037616248
24	20	5	0.1	20	0.119970717	0.325216536	0.3509325	0.036405253
24	20	5	0.5	16	0.117808723	0.340731834	0.328150171	0.039459335
24	20	5	0.5	18	0.112246438	0.35286297	0.359558867	0.059869666
24	20	5	0.5	20	0.119970717	0.325216536	0.333398912	0.050406599
24	20	6	0	18	0.112246438	0.35286297	0.269598007	0.044577703
24	20	6	0.1	18	0.112246438	0.35286297	0.426802929	0.045313946
24	20	6	0.5	16	0.117808723	0.340731834	0.315422046	0.065343015
24	20	6	0.5	18	0.112246438	0.35286297	0.348243961	0.062207036
24	20	6	0.5	20	0.119970717	0.325216536	0.317355156	0.049740282
24	20	7	0	18	0.112246438	0.35286297	0.657100766	0.049356291
24	20	7	0.1	18	0.112246438	0.35286297	0.313276536	0.051035064
24	20	7	0.5	16	0.117808723	0.340731834	0.312013342	0.054864899
24	20	7	0.5	18	0.112246438	0.35286297	0.337360905	0.063688997
24	20	7	0.5	20	0.119970717	0.325216536	0.33993197	0.052544791
24	23	5	0	16	0.130595776	0.347522233	0.230059712	0.03533942
24	23	5	0	18	0.151315067	0.357331214	0.418150599	0.036191662
24	23	5	0	20	0.124401612	0.322860066	0.386483148	0.033891592
24	23	5	0.1	16	0.130595776	0.347522233	0.478601332	0.033823258
24	23	5	0.1	18	0.151315067	0.357331214	0.370274081	0.034250916
24	23	5	0.1	20	0.124401612	0.322860066	0.268966062	0.034260235
24	23	5	0.5	16	0.130595776	0.347522233	0.304883196	0.046709724
24	23	5	0.5	18	0.151315067	0.357331214	0.421513869	0.043543589
24	23	5	0.5	20	0.124401612	0.322860066	0.275943713	0.041427315
24	23	6	0	16	0.130595776	0.347522233	0.226208074	0.03979458
24	23	6	0	18	0.151315067	0.357331214	0.314892239	0.03988182
24	23	6	0	20	0.124401612	0.322860066	0.232297002	0.039541622
24	23	6	0.1	16	0.130595776	0.347522233	0.216309413	0.039182483
24	23	6	0.1	18	0.151315067	0.357331214	0.351560575	0.039662168
24	23	6	0.1	20	0.124401612	0.322860066	0.362003572	0.040611839
24	23	6	0.5	16	0.130595776	0.347522233	0.39750474	0.04955008
24	23	6	0.5	18	0.151315067	0.357331214	0.314955168	0.047159018
24	23	6	0.5	20	0.124401612	0.322860066	0.456307151	0.045442126
24	23	7	0	16	0.130595776	0.347522233	0.268820131	0.046337355
24	23	7	0	18	0.151315067	0.357331214	0.359922352	0.046114273
24	23	7	0	20	0.124401612	0.322860066	0.291952887	0.046591245
24	23	7	0.1	16	0.130595776	0.347522233	0.255361776	0.044700053
24	23	7	0.1	18	0.151315067	0.357331214	0.28759481	0.04565009
24	23	7	0.1	20	0.124401612	0.322860066	0.287294438	0.045792016
24	23	7	0.5	16	0.130595776	0.347522233	0.280597023	0.065463193
24	23	7	0.5	18	0.151315067	0.357331214	0.292524491	0.055200909
24	23	7	0.5	20	0.124401612	0.322860066	0.284647096	0.061282099
24	25	5	0	16	0.118254842	0.347724685	0.268768734	0.033182169
24	25	5	0	18	0.120276451	0.340963158	0.429839291	0.034093891
24	25	5	0	20	0.122224145	0.317708799	0.241971475	0.033895628
24	25	5	0.1	16	0.118254842	0.347724685	0.286629756	0.033402023
24	25	5	0.1	18	0.120276451	0.340963158	0.318693356	0.034098156

LB	BS	NEU	TH	NF	Train	Static	Online	Time
24	25	5	0.1	20	0.122224145	0.317708799	0.286263883	0.034194557
24	25	5	0.5	16	0.118254842	0.347724685	0.295697346	0.040801044
24	25	5	0.5	18	0.120276451	0.340963158	0.293781317	0.04291352
24	25	5	0.5	20	0.122224145	0.317708799	0.387354598	0.036719504
24	25	6	0	18	0.120276451	0.340963158	0.270335952	0.040067021
24	25	6	0.1	18	0.120276451	0.340963158	0.375185863	0.041718137
24	25	6	0.5	16	0.118254842	0.347724685	0.353371657	0.054683735
24	25	6	0.5	18	0.120276451	0.340963158	0.32066527	0.059417118
24	25	6	0.5	20	0.122224145	0.317708799	0.275927944	0.062535562
24	25	7	0	18	0.120276451	0.340963158	0.415482773	0.064440644
24	25	7	0.1	18	0.120276451	0.340963158	0.269077588	0.064031578
24	25	7	0.5	16	0.118254842	0.347724685	0.487295061	0.068174794
24	25	7	0.5	18	0.120276451	0.340963158	0.274269481	0.081909689
24	25	7	0.5	20	0.122224145	0.317708799	0.376445479	0.055878223

4.1 Mejoras de velocidad

Los resultados de la Tabla 7 fueron obtenidos tras modificar el código de la red CNN+LSTM para aumentar la velocidad de procesamiento. La primera mejora pasa por evitar que Keras tenga que reorganizar la memoria de la GPU cada vez que se llama a la función `.fit()`. Esto se evita utilizando un `Callback` en la llamada a la función `.fit()`, forzando a Keras a organizar la memoria de la GPU una sola vez, mandando a la GPU la orden de realizar todas las iteraciones del bucle en una sola llamada. Así, el bucle que llama a la función `.fit()` un total de `n_epochs` veces queda de la forma:

```

1 #Callback para la funcion .fit()
2 class ResetStatesCallback(tf.keras.callbacks.Callback):
3     def on_epoch_end(self, epoch, logs=None):
4         for layer in self.model.layers:
5             if hasattr(layer, 'reset_states'):
6                 layer.reset_states()
7
8
9 #El bucle se transforma en una unica llamada a la funcion .fit()
10 myCNNLSTM.fit(
11     trainX, trainY,
12     epochs=n_epochs,
13     batch_size=batch_size,
14     verbose=0,
15     shuffle=True,
16     callbacks=[ResetStatesCallback()] # Callback para .fit()
17 )

```

Listing 13: Callback para la función `.fit()`

La segunda mejora se encuentra en la parte de entrenamiento incremental. También está basada en evitar que el intérprete de Python tenga que hacer demasiadas llamadas a la GPU. En este caso he invocado al compilador `AutoGraph` mediante `@tf.function`, que transforma el código de Python en operaciones matemáticas optimizadas de `Tensorflow`. Además, utiliza `Accelerated Linear Algebra (XLA)` y `Kernel Fusion` para simplificar las operaciones matemáticas en un único comando para la GPU.

```

1 @tf.function #Invocar AutoGraph, XLA y Kernel Fusion
2 def skima_train_step(model, optimizer, x, y, norm_sq):
3     with tf.GradientTape() as tape:
4         predictions = model(x, training=True)
5         loss = rmse_loss(y, predictions)
6
7     gradients = tape.gradient(loss, model.trainable_variables)

```

```

8   adjuted_gradients = [g * norm_sq for g in gradients]
9   optimizer.apply_gradients(zip(adjuted_gradients, model.trainable_variables))
10
11
12  # Uso de la funcion skima_train_step()
13  if rmse_norm > error_threshold:
14      st = tt.time()
15
16      #RMSE_NORM como tensor para AutoGraph
17      norm_sq_tensor = tf.cast(rmse_norm**2, tf.float32)
18
19      for j in range(n_epochs_update):
20          # Uso de XLA y Kernel Fusion
21          skima_train_step(myCNNLSTM, optimizer, online_X, online_Y, norm_sq_tensor)
22
23      # Resetear los estados
24      for layer in myCNNLSTM.layers:
25          if hasattr(layer, 'reset_states'):
26              layer.reset_states()
27
28      et = tt.time()
29      .
30      .
31      .

```

Listing 14: Uso de AutoGraph, XLA y Kernel Fusion mediante @tf.function

El último Grid Search, descrito en la Tabla 7, se realizó con las mejoras de velocidad aquí descritas. Tardó 43200.4 segundos en procesar todas sus combinaciones. Por su parte, el Grid Search anterior, descrito en la Tabla 6, tardó 34459.3 segundos. Las mejoras introducidas merecen mucho la pena, como se puede observar en la columna **Time** de la Tabla 7.

$$43200.4 / (3 * 3 * 3 * 3 * 3 - 2 * 2 * 2 * 2 * 2) = 204.74 \text{ segundos/combinacion con mejora}$$

$$34459.3 / (1 * 3 * 3 * 3 * 3) = 1076.85 \text{ segundos/combinacion sin mejora}$$

Para comprobar que estas mejoras de velocidad no hubiesen comprometido la reproducibilidad, he realizado de nuevo una parte del último Grid Search, detallado en la Tabla 8 y he comprobado que la reproducibilidad se ha visto afectada en lo más mínimo.

Table 8: Resultados de la prueba de reproducibilidad con Grid Search extendido 7x7 CNN+LSTM con mejoras de velocidad (LB=lookback BS=batchsize NEU=n_epoch_updates TH=threshold NF=número filtros)

LB	BS	NEU	TH	NF	Train	Static	Online	Time
18	20	5	0	16	0.124188291	0.351130706	0.44549564	0.031099011
18	20	5	0	18	0.125561637	0.34379394	0.279698489	0.03188416
18	20	5	0	20	0.157130497	0.333308253	0.289715531	0.031903756
18	20	5	0.1	16	0.124188291	0.351130706	0.337810675	0.031468513
18	20	5	0.1	18	0.125561637	0.34379394	0.324740071	0.03163094
18	20	5	0.1	20	0.157130497	0.333308253	0.557908275	0.031265837
18	20	5	0.5	16	0.124188291	0.351130706	0.261651886	0.041009765
18	20	5	0.5	18	0.125561637	0.34379394	0.317533034	0.036896685
18	20	5	0.5	20	0.157130497	0.333308253	0.280227668	0.042221343
18	20	6	0	16	0.124188291	0.351130706	0.22750189	0.037775457
18	20	6	0	18	0.125561637	0.34379394	0.330851318	0.038064967
18	20	6	0	20	0.157130497	0.333308253	0.279271202	0.037631272

LB	BS	NEU	TH	NF	Train	Static	Online	Time
18	20	6	0.1	16	0.124188291	0.351130706	0.459342456	0.037110539
18	20	6	0.1	18	0.125561637	0.34379394	0.401013509	0.038228965
18	20	6	0.1	20	0.157130497	0.333308253	0.299093759	0.037564812
18	20	6	0.5	16	0.124188291	0.351130706	0.303918994	0.04674291
18	20	6	0.5	18	0.125561637	0.34379394	0.39852485	0.040692408
18	20	6	0.5	20	0.157130497	0.333308253	0.263999611	0.050903508
18	20	7	0	16	0.124188291	0.351130706	0.270392779	0.043852834
18	20	7	0	18	0.125561637	0.34379394	0.36236949	0.044185738
18	20	7	0	20	0.157130497	0.333308253	0.264339883	0.043160081
18	20	7	0.1	16	0.124188291	0.351130706	0.328284274	0.043354338
18	20	7	0.1	18	0.125561637	0.34379394	0.242497093	0.046209212
18	20	7	0.1	20	0.157130497	0.333308253	0.342248843	0.043852427
18	20	7	0.5	16	0.124188291	0.351130706	0.275108437	0.047630396
18	20	7	0.5	18	0.125561637	0.34379394	0.451926269	0.044914621
18	20	7	0.5	20	0.157130497	0.333308253	0.276592488	0.05450598
18	23	5	0	16	0.132810299	0.33521819	0.367564029	0.031903366
18	23	5	0	18	0.128073586	0.32483796	0.615401825	0.032980608
18	23	5	0	20	0.126368559	0.317677541	0.472691303	0.03251948
18	23	5	0.1	16	0.132810299	0.33521819	0.27981442	0.03248245
18	23	5	0.1	18	0.128073586	0.32483796	0.326615437	0.03300204
18	23	5	0.1	20	0.126368559	0.317677541	0.408602846	0.032937142
18	23	5	0.5	16	0.132810299	0.33521819	0.355172789	0.035728255
18	23	5	0.5	18	0.128073586	0.32483796	0.407765671	0.037294768
18	23	5	0.5	20	0.126368559	0.317677541	0.274672361	0.038190356
18	23	6	0	16	0.132810299	0.33521819	0.286590484	0.03778299
18	23	6	0	18	0.128073586	0.32483796	0.273141048	0.038384681
18	23	6	0	20	0.126368559	0.317677541	0.504135745	0.038898348
18	23	6	0.1	16	0.132810299	0.33521819	0.268178265	0.038251911
18	23	6	0.1	18	0.128073586	0.32483796	0.293402615	0.039148848
18	23	6	0.1	20	0.126368559	0.317677541	0.246195606	0.03882059
18	23	6	0.5	16	0.132810299	0.33521819	0.334052626	0.040808667
18	23	6	0.5	18	0.128073586	0.32483796	0.297844432	0.043127096
18	23	6	0.5	20	0.126368559	0.317677541	0.284678399	0.047929206
18	23	7	0	16	0.132810299	0.33521819	0.261624957	0.043860082
18	23	7	0	18	0.128073586	0.32483796	0.28099613	0.045822698
18	23	7	0	20	0.126368559	0.317677541	0.247007135	0.044874667
18	23	7	0.1	16	0.132810299	0.33521819	0.311369425	0.044143038
18	23	7	0.1	18	0.128073586	0.32483796	0.323032644	0.044961265
18	23	7	0.1	20	0.126368559	0.317677541	0.303077048	0.043615721
18	23	7	0.5	16	0.132810299	0.33521819	0.336221312	0.049867789
18	23	7	0.5	18	0.128073586	0.32483796	0.409013863	0.052465135
18	23	7	0.5	20	0.126368559	0.317677541	0.452651562	0.044780894
18	25	5	0	16	0.140966651	0.317910143	0.279664385	0.031828402
18	25	5	0	18	0.125616867	0.32494159	0.300975342	0.034751067
18	25	5	0	20	0.134359081	0.3193303	0.452861752	0.032876158
18	25	5	0.1	16	0.140966651	0.317910143	0.244987354	0.032459684
18	25	5	0.1	18	0.125616867	0.32494159	0.262777857	0.033170379
18	25	5	0.1	20	0.134359081	0.3193303	0.246148344	0.033473293
18	25	5	0.5	16	0.140966651	0.317910143	0.271096583	0.038503606
18	25	5	0.5	18	0.125616867	0.32494159	0.332152581	0.037681561
18	25	5	0.5	20	0.134359081	0.3193303	0.347808418	0.035772088
18	25	6	0	16	0.140966651	0.317910143	0.304314092	0.038155409
18	25	6	0	18	0.125616867	0.32494159	0.227776985	0.039165414
18	25	6	0	20	0.134359081	0.3193303	0.460571397	0.038780237
18	25	6	0.1	16	0.140966651	0.317910143	0.279490356	0.037954308

LB	BS	NEU	TH	NF	Train	Static	Online	Time
18	25	6	0.1	18	0.125616867	0.32494159	0.32650844	0.03869829
18	25	6	0.1	20	0.134359081	0.3193303	0.327456883	0.038777084
18	25	6	0.5	16	0.140966651	0.317910143	0.274607351	0.04310953
18	25	6	0.5	18	0.125616867	0.32494159	0.273795625	0.043918527
18	25	6	0.5	20	0.134359081	0.3193303	0.29187384	0.045936941
18	25	7	0	16	0.140966651	0.317910143	0.269967791	0.043469509
18	25	7	0	18	0.125616867	0.32494159	0.259485365	0.044977944
18	25	7	0	20	0.134359081	0.3193303	0.274488607	0.045054096
18	25	7	0.1	16	0.140966651	0.317910143	0.278330354	0.044080902
18	25	7	0.1	18	0.125616867	0.32494159	0.311502188	0.044780315
18	25	7	0.1	20	0.134359081	0.3193303	0.33766317	0.044725981
18	25	7	0.5	16	0.140966651	0.317910143	0.267004107	0.052661598
18	25	7	0.5	18	0.125616867	0.32494159	0.299357134	0.050753234
18	25	7	0.5	20	0.134359081	0.3193303	0.348501788	0.050514755

5 Resultados CNN+CNN+LSTM

He aumentado el número de capas de la red neuronal CNN y probado con el siguiente set de 972 combinaciones de hiperparámetros:

```

1  #Parameter tweak
2  hyperparameters = {
3      'look_back': [18, 20, 24],
4      'batch_size': [23, 25, 27],
5      'n_epochs_update': [5, 6, 7],
6      'threshold': [0, 0.1, 0.15, 0.2],
7      'num_filters1': [12, 14, 16],
8      'num_filters2': [20, 22, 24]
9  }

```

Listing 15: Grid Search CNN+CNN+LSTM reproducible

Siendo `num_filters1` el número de filtros de la primera capa y `num_filters2` el número de filtros de la segunda capa. Visto que la primera capa analiza los datos para extraer patrones básicos, he considerado darle menos filtros que a la segunda capa, que debe transformar los patrones básicos en patrones más complejos.

Los mejores resultados en cuanto a `Avg_Test_Inc_Score` fueron:

Table 9: Mejor combinación para la red CNN+CNN+LSTM

LB	BS	NEU	TH	NF1	NF2	Train	Static	Online	Time
18	25	6	0.1	14	20	0.122076	0.366933	0.211242	0.065284

5.1 Análisis de resultados

Al tener tantos resultados, no es posible analizarlos a mano. Por ello, para poder identificar las tendencias presentes en los datos, he utilizado el coeficiente de Spearman.

Este coeficiente permite encontrar tendencias en los datos sin necesidad de que estas tendencias sean estrictamente lineales como ocurre con el coeficiente de Pearson. Además, hay suficientes datos como

para que se desvanezcan las ventajas del coeficiente de Kendall. Además, el coeficiente de Spearman puede indicar correlaciones proporcionales (signo positivo) o inversamente proporcionales (signo negativo).

En este caso, un signo negativo indica que el error o el tiempo de procesamiento disminuye cuando el hiperparámetro en cuestión aumenta y viceversa.

Table 10: Coeficientes de Correlación de Spearman para CNN+CNN+LSTM

	AvgTrainRMSE	AvgTestRMSE	AvgTestIncRMSE	AvgTestIncTime
lookback	-0.121340	0.148394	-0.037053	-0.010026
batchsize	0.266828	-0.047088	0.018862	-0.275763
updates	-0.020447	0.007675	-0.150139	0.828263
threshold	0.022866	0.007318	-0.003070	-0.030932
numfilters1	-0.077155	0.028489	-0.093504	0.046299
numfilters2	-0.345310	-0.156740	-0.046374	0.032351

Un coeficiente de Spearman menor a 0.05 es considerado como una ausencia de correlación. Así, los hiperparámetros que influyen en el rendimiento del entrenamiento incremental son **n_epoch_updates** y **num_filters1**. Tiene sentido cuantas más veces se reentrene la red neuronal con el nuevo dato obtenido (**n_epoch_updates**), menor sea el error puesto que la red neuronal tiene más posibilidades de adaptarse al nuevo dato. También, al añadir más filtros en la primera capa CNN, la red puede extraer más patrones y usarlos para las predicciones. Es interesante ver que el número de filtros de la segunda capa no tiene un impacto predecible en el entrenamiento incremental.

Para el test convencional, los hiperparámetros con una tendencia visible son el número de muestras que la red procesa al mismo tiempo y el número de filtros de la segunda capa CNN. No encuentro mucha justificación a esto. Porbablemente al subir el número de muestras (ventana) que la red procesa a un mismo tiempo haga que la red no extraiga bien el patrón de dicha ventana pero no es una explicación plausible con la correlación inversa que tiene **look_back** con el error de entrenamiento. Otra explicación probable es que aumentar el tamaño de la ventana provoque un pequeño efecto de *overfitting*, lo que explicaría la relación inversa con el error de entrenamiento y la relación directa con el error de test. Respecto a los filtros de la segunda capa tampoco tengo una explicación coherente. ¿Por qué los filtros de la segunda sí y los de la primera no? ¿Por qué en el entrenamiento incremental es al revés? Sinceramente no tengo respuesta todavía. De todas formas, tienen coeficientes de Spearman relativamente bajos.

Respecto al entrenamiento, un aumento del tamaño de los *batches* produce una caída del rendimiento, seguramente porque al aumentar el tamaño de los batches hay menos batches en el dataset y el entrenamiento es menos intensivo. Con el número de filtros de las capas CNN pasa lo mismo que con el rendimiento en test aunque esta vez un coeficiente de Spearman de -0.345 no es despreciable.

Respecto al tiempo de procesamiento del entrenamiento incremental tiene unos coeficientes de Spearman previsibles. Al aumentar el número de veces que se reentrena la red con un mismo dato (**n_epoch_update**) aumenta el tiempo que tarda. Cando aumenta el tamaño del *batch*, las GPU pueden procesar más datos en paralelo. En cuanto al hiperparámetro **threshold**, hay que tener en cuenta que sólo se computan y se tienen en cuenta los tiempos en los cuales el error de la red es mayor al **threshold** y se ejecuta el entrenamiento incremental. Cuando el **threshold** no permite que se realice el entrenamiento incremental el tiempo de ejecución **TestIncTime** es 0 pero no se tiene en cuenta para el **AvgTestIncTime** puesto que daría resultados muy cercanos a cero y se perdería la información de el tiempo que realmente tarda el entrenamiento incremental en asumir un dato nuevo.

6 Resultados CNN+CNN+CNN+LSTM

También he realizado pruebas con 3 capas de CNN. He probado con el siguiente set de combinaciones de hiperparámetros:

```
1 #Parameter tweak
2 hyperparameters = {
3     'look_back': [18, 24],
4     'batch_size': [23, 25],
5     'n_epochs_update': [5, 6, 7],
6     'threshold': [0, 0.1],
7     'num_filters1': [12, 14, 16],
8     'num_filters2': [20, 22, 24],
9     'num_filters3': [26, 28, 30]
10 }
```

Listing 16: Grid Search CNN+CNN+CNN+LSTM

Todavía no he completado el *Grid search* puesto que tarda mucho en procesar pero por ahora el ganador es:

Table 11: Mejor combinación para la red CNN+CNN+LSTM

LB	BS	NEU	TH	NF1	NF2	NF3	Train	Static	Online	Time
18	25	6	0	12	24	28	0.11261	0.32942	0.213898	0.073121

Aunque hay una capa más de CNN, los resultados no han sido mejores que con sólo 2 capas aunque están cerca. Además de ser una red más compleja y costosa de entrenar.

No creo que añadir una cuarta capa sea beneficioso para un *grid* de 7x7. Con un *kernel* de 3x3, la cuarta capa sólo necesitaría una celda para recoger información de todas las 49 celdas al mismo tiempo.

Tampoco creo que añadir capas de *pooling* sea algo beneficioso. Después de todo, un *grid* de 7x7 es relativamente pequeño y una capa de *pooling* puede dejar a la red neuronal sin datos. Además, añadir capas de CNN tiene la intención de proporcionar a la red datos acerca de las relaciones espaciales entre las celdas. Si añado una capa de *pooling* estoy destruyendo esa información.

7 Grid 15x15 y kernel 2x2

He creado el dataset para 15x15 celdas a partir del dataset Milán y probado con kernels tanto de 2x2 como de 3x3. He realizado también un análisis de Spearman en todo el conjunto de experimentos para intentar vislumbrar la relación entre los hiperparámetros y los resultados. El análisis indica que el hiperparámetro más influyente es la cantidad de veces que la red neuronal ajusta sus valores en cada vuelta al bucle de entrenamiento incremental (`n_epochs.update`). Al aumentar `n_epochs.update` disminuye el error en el entrenamiento incremental pero aumenta mucho el tiempo de ejecución. También, aunque en menor medida, influyen de la misma forma `look_back` y `batch_size`. Viendo esto he realizado algunas pruebas con valores altos de estas variables.

7.1 Resultados

En general un kernel 2x2 ha mejorado los resultados obtenidos hasta el momento para un grid de 7x7. Aun así, los resultados siguen sin ser satisfactorios puesto que el error de entrenamiento incremental sigue por encima del 20%.

Los resultados con el grid 15x15 han sido fatídicos tanto con un kernel 2x2 como un kernel 3x3. El mejor resultado ha sido 22.617% en AvgTestIncRNMSE.

Las pruebas con valores altos de `n_epoch_updates` han dado los mejores resultados hasta el momento (20.4%) en entrenamiento incremental pero aumentan mucho el tiempo de ejecución. Aun así, siguen siendo malos resultados al no bajar del 20%.